"Sirf Basic LWC Aata Hai?"

Toh 3-5 YOE Salesforce Developer Interview Mein Rejection Almost Confirm Hai.

Aajkal MNCs sirf component creation nahi...
Real-time LWC architecture, Apex integration, async handling, performance optimization aur production debugging check karti hain.

Below are 90+ toughest Scenario-Based LWC Interview Questions asked in enterprise-level interviews.

Questions Prepared By – Prominent Academy

Advanced LWC Architecture

- Component communication, Composition, Dynamic Creation, Slot, Template Optimization

Apex Integration

- Imperative vs @wire, Caching, Error Handling, Complex Data Mapping

Async Handling

- Promises, async/await, Chained Calls, Parallel Execution, Error Handling

Performance Optimization

- Minimize Re-renders, Efficient Data Handling, Lazy Loading, Memory Management

Production Debugging

- Debugging in Browser, Logs, Common Issues, Locker Service, CSP, CORS

Security & Best Practices

- CRUD/FLS, Data Security, XSS Protection, Secure Apex Calls

Don't just create components,
Build enterprise-ready LWC solutions!

Salesforce LWC

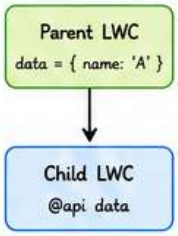
Scenario-Based Interview Questions (3-5 YOE)

1 You created an LWC component but data is not rendering after deployment in Production. It works in Sandbox. How will you debug?

Expected Discussion Areas :

- Check browser console errors
- Verify Apex class/method access (FLS, CRUD, Sharing)
- Check CSP (Content Security Policy) issues
- Validate Named Credentials / Endpoints
- Use Lightning Inspector (LWC tab) to inspect data
- Check org differences (Custom Metadata, Settings)
- Add logs, try/catch in Apex and handle errors in UI

2 A parent component sends data to child, but child UI doesn't refresh after value change. Why?



Expected Discussion Areas :

- Pass by reference – object mutated but not reassigning
- LWC doesn't track deep object changes
- You are not using @track for complex object
- Child not reactive to primitive change
- Fix: Reassign new object/array reference or use @track / @api with proper updates

3 How would you design a reusable generic LWC component used across 15 different modules?

Expected Discussion Areas :

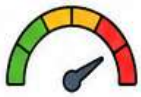
- Make it configurable via @api properties
- Use slots for flexible UI sections
- Follow composition over inheritance
- Keep business logic out – purely UI/UX component
- Support events for communication
- Document with jsDocs and storybook
- Ensure responsive and accessible design

4 When would you prefer Aura over LWC in real projects?

Expected Discussion Areas :

- When using older features not yet in LWC
- Requirements need $A, aura:if, aura:iteration, etc.
- Heavy dependency on Lightning Component framework features
- Need to support IE11 (legacy orgs)
- Existing Aura component with heavy investment
- Complex third-party JS libs already in Aura

5 How do you optimize initial loading performance of heavy LWC pages?



Expected Discussion Areas :

- Lazy load components (dynamic import)
- Use @wire with cacheable Apex methods
- Avoid unnecessary @wire calls
- Break page into tabs/accordions/lazy sections
- Use LDS (Lightning Data Service) where possible
- Minimize DOM size and nested components
- Optimize images, use static resources and CDNs

6 Your LWC component works slowly when 10k records load. What architecture changes would you do?

Expected Discussion Areas :

- Implement Server-Side Pagination
- Use Data Table with infinite loading
- Avoid fetching all records – query selective fields
- Use Search/Filter to reduce dataset
- Virtual scrolling for large lists
- Move heavy processing to Apex
- Cache frequently used data (Platform Cache)

7 How do you prevent unnecessary re-rendering in LWC?

Expected Discussion Areas :

- Use immutable data patterns
- Reassign new object/array reference only when data actually changes
- Avoid inline functions in template
- Use getter wisely – avoid heavy computation
- Use key in for:each (key={item.Id})
- Don't mutate objects inside template
- Use conditional rendering wisely

8 What happens internally when @track property changes?

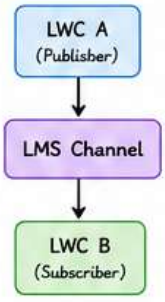
Expected Discussion Areas :

- LWC observes changes via Proxy (reactive membrane)
- Change triggers component reactivity
- Virtual DOM diffing happens
- Only changed parts are updated in real DOM
- Lifecycle hooks (renderedCallback) may run
- Dependent getters and templates re-evaluate

9 Difference between reactive and non-reactive properties in LWC?

Reactive (@track / @wire / @api)	Non-Reactive
LWC tracks changes automatically	Changes won't trigger re-render
UI updates when value changes	UI remains stale
Used with primitives, objects, arrays, @wire data	Used with non-tracked objects, plain variables
Needs new reference for objects and arrays	Changing internal value won't reflect in UI

10 You need component communication between unrelated LWCs. How will you implement?



Expected Discussion Areas :

- Use Lightning Message Service (LMS) – recommended (publish/subscribe across components)
- Use custom events on a common ancestor component
- Use Platform Events (for real-time org-wide needs)
- Use Application State or Custom Store (for large scale)
- Avoid using window/global variables

Pro Tip: Focus on reusability, performance, scalability & clean architecture in LWC development. Great LWC developers build solutions that are fast, maintainable and business-focused.

Lifecycle Hooks Scenarios

11 You're calling Apex inside connectedCallback() but API gets invoked multiple times unexpectedly. Why?

Expected Discussion Areas :

- connectedCallback() can fire multiple times (DOM re-insertion, conditional rendering)
- Component gets destroyed & recreated
- Parent re-render causes child re-initialization
- Use a flag to prevent duplicate API calls
- Move logic to wire where possible
- Use caching / store result to avoid re-fetching

12 When should renderedCallback() never be used?

Expected Discussion Areas :

- Avoid DML operations
- Don't call Apex repeatedly from renderedCallback()
- Don't update tracked properties inside it (causes infinite loop)
- Don't manipulate data-heavy DOM repeatedly
- Not for data fetching – use connectedCallback()
- Avoid performance-heavy logic (runs after every render)

13 A renderedCallback() causes infinite loop in Production. Explain root cause.

Expected Discussion Areas :

- Updating a @track property inside renderedCallback()
- That update triggers re-render
- renderedCallback() runs again → loop continues
- Fix: Move logic to connectedCallback() or use condition
- Add flags to run code only once
- Avoid DOM updates that trigger re-render

14 How do lifecycle hooks execute during navigation between Lightning pages?

Expected Discussion Areas :

- New page → new component instance
- connectedCallback() fires when component is inserted into DOM
- renderedCallback() fires after each render
- Navigating away → disconnectedCallback() fires
- Returning back may create new instance (no state)
- Use caching (Lightning Data Service / session storage) to preserve state

15 A child component loads before parent data arrives. How will you handle?

Expected Discussion Areas :

- Parent passes data using @api property
- Child should handle undefined/null data
- Use getter + conditional rendering (if:true)
- Show skeleton / loader until data available
- Use @wire in child if data is independent
- Dispatch event from child to notify parent if needed

16 Difference between connectedCallback() and renderedCallback() in practical usage?

Expected Discussion Areas :

connectedCallback()	renderedCallback()
Called once when component is inserted into DOM	Called after every render
Use for data fetch, event listeners, initialization	Use for DOM manipulation, focus, measurements
Ideal place for Apex calls (wire or imperative)	Runs multiple times – heavy logic causes performance issue
Not dependent on template render cycle	Can cause infinite loop if updating tracked properties

17 How do you cleanup event listeners in disconnectedCallback()?

Expected Discussion Areas :

- Remove event listeners added in connectedCallback()
- Use removeEventListener() for custom events
- Unsubscribe from LMS / Emp API / CometD
- Clear intervals/timeouts
- Disconnect observers (ResizeObserver, MutationObserver)
- Avoid memory leaks and unintended behavior

18 A component keeps old cached values even after refresh. Which lifecycle issue could cause this?

Expected Discussion Areas :

- Data cached in module-level variable (persists across instances)
- Not resetting values in connectedCallback()
- Using static variables or singleton pattern incorrectly
- Not clearing LDS cache (refreshApex / notifyRecordUpdateAvailable)
- Conditional rendering causing component reuse
- Fix: Reset state in connectedCallback() or use proper caching strategy

19 How would you delay rendering until all async operations complete?

Expected Discussion Areas :

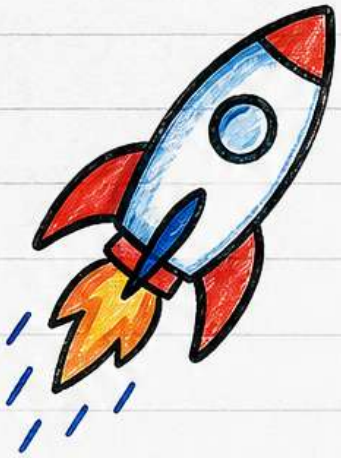
- Use boolean flag (isLoaded / isReady)
- Call async operations in connectedCallback()
- Use Promise.all() for multiple async calls
- Show spinner until all complete
- Set flag = true and render content using if:true
- Avoid rendering incomplete UI

20 How do you detect whether component is rendered first time or rerendered?

Expected Discussion Areas :

- Use a flag variable (e.g., isFirstRender = true)
- In renderedCallback(), check flag
- If true → first render, then set to false
- Else → it's a re-render
- Helps in DOM manipulation, focusing, one-time initialization

Pro Tip: Understand when each lifecycle hook runs and keep your logic in the right place. It improves performance, avoids bugs, and builds scalable LWC components!



Prominent Academy

- Real-time LWC mock interviews
- Advanced Apex + LWC preparation
- Enterprise-level scenario discussions
- Production-level debugging scenarios

salesforce

Pay After Placement

+91 93594 45862

Salesforce interviews ab sirf syntax nahi check karte.
They test real-time problem solving, architecture thinking & production-level decision making.

#SalesforceDeveloper #LWC #LightningWebComponents
#SalesforceInterview #Apex #AsyncApex
#SalesforceJobs #TechInterview #ProminentAcademy

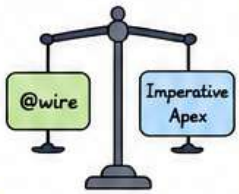
Apex Integration Scenarios

Scenario-Based Interview Questions (3-5 YOE)

21 When would you use @wire vs imperative Apex calls?

Expected Discussion Areas :

- @wire for read-only, cacheable data
- Automatic data provisioning & reactivity
- Imperative for DML, non-cacheable, conditional or on-demand actions
- @wire has limitations (no parameters change, no control over timing)
- Performance and best practices



22 A wired method returns stale data. How will you refresh dynamically?

Expected Discussion Areas :

- Use refreshApex() with wired result
- Use Lightning Data Service refresh
- Cacheable method considerations
- Re-fetch after DML operations
- Tradeoff between performance and data freshness

23 How do you handle Apex exceptions gracefully inside LWC?

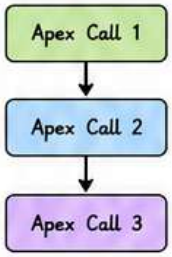
Expected Discussion Areas :

- try-catch with async/await
- Handle AuraHandledException
- Show meaningful user messages
- Log errors (custom logging framework)
- Differentiate between handled & unhandled exceptions

24 You need sequential Apex calls dependent on previous response. How will you design?

Expected Discussion Areas :

- Use async/await for sequential flow
- Promise chaining approach
- Handle errors at each step
- Maintain state between calls
- Show loading indicators per step



25 How do you avoid hitting Apex governor limits from LWC?

Expected Discussion Areas :

- Bulkify operations
- Minimize number of Apex calls
- Use batching and pagination
- Avoid SOQL in loops, reduce DML
- Use selective queries and indexed fields
- Optimized data transfer (return needed fields only)

26 Your Apex response contains huge JSON payload causing UI lag. Optimization approach?

Expected Discussion Areas :

- Return only required fields
- Pagination / lazy loading
- Compress or simplify response
- Use wrapper classes
- Virtual scrolling for large lists
- Avoid deep nested JSON structures

27 How do you implement retry mechanism for failed Apex calls?

Expected Discussion Areas :

- Retry with exponential backoff
- Limit retry attempts
- Handle specific exception types
- Queue for later processing
- Show user feedback and status
- Use Platform Events for async retry

28 Apex method returns null unexpectedly in Production only. Debugging steps?

Expected Discussion Areas :

- Check data differences (profiles, sharing)
- Verify field-level security
- Enable debug logs in Production
- Validate SOQL queries and filters
- Check return conditions in Apex
- Add system.debug logs and monitor in real-time

29 How do you secure Apex methods exposed to LWC?

Expected Discussion Areas :

- Use with sharing
- Enforce CRUD / FLS
- Validate input parameters
- Use sharing rules and restriction rules
- Avoid SOQL injection (bind variables)
- Check user permissions before operations

30 You need real-time updates after database change without page refresh. How?

Expected Discussion Areas :

- Use Lightning Message Service (LMS)
- Platform Events + EMP API
- Change Data Capture (CDC)
- Streaming API for real-time updates
- Subscribe and react in LWC
- Push vs Polling comparison

Pro Tip: Design for performance, security, scalability and user experience while integrating Apex with LWC.

Lightning Data Service (LDS) Scenarios

31 When should LDS be preferred over Apex?

Expected Discussion Areas :

- Use LDS for UI data operations on standard objects
- When using Lightning Base Components (e.g., record-form, record-view-form)
- For better performance, caching & offline support
- When CRUD/FLS/sharing enforcement is required automatically
- When real-time data sync across components is needed
- Avoid Apex for simple CRUD to reduce server calls

32 Your record form doesn't show updated values immediately. Why?

Expected Discussion Areas :

- LDS cache not refreshed
- Record updated outside current component
- Missing RefreshView API / notifyRecordUpdateAvailable
- Stale data due to offline cache
- Using @wire with cacheable Apex instead of LDS
- Same record opened in multiple tabs/components

33 How does LDS improve performance internally?

Expected Discussion Areas :

- Client-side caching reduces server round trips
- Smart data prefetching & record hydration
- Data sharing across components (single source)
- Optimistic UI updates
- Delta updates instead of full payload
- Built-in batching & background sync

34 A record edit form fails for some profiles only. Root cause possibilities?

Expected Discussion Areas :

- FLS (Field Level Security) restrictions
- Object/Field permissions missing
- Record Type restrictions
- Page Layout doesn't include the fields
- Sharing rules / OWD / Role hierarchy
- Validation rules or Apex triggers failing

35 How do you implement custom validation with LDS?

Expected Discussion Areas :

- Use Lightning Data Service UI API errors
- Validation Rules on object/fields
- Apex Trigger validations (addsError)
- Client-side validation before calling submit()
- Use lightning-input-field + reportValidity()
- Show errors using lightning-messages component

36 Can LDS bypass sharing/security? Explain.

Expected Discussion Areas :

- No, LDS enforces CRUD, FLS, Sharing automatically
- Uses User mode under the hood
- Cannot bypass sharing with LDS
- For admin-level operations, Apex with "with sharing" / "without sharing" is required
- LDS respects org-wide security model

37 How do you refresh LDS data after custom save operation?

Expected Discussion Areas :

- Use notifyRecordUpdateAvailable(recordIds)
- Use RefreshView API (refreshApex for @wire)
- Use getRecordNotifyChange(recordIds)
- Navigate away and back (forces reload)
- Dispatch custom event to refresh parent components
- Avoid full page refresh (forces re-render)

38 What are LDS limitations in enterprise implementations?

Expected Discussion Areas :

- Works only with supported standard & custom objects
- No support for complex joins/aggregations
- Limited control over SOQL queries
- Not ideal for large data volume operations
- No long-running transactions
- UI API limits apply (records/fields per request)

39 How do you handle dependent picklists in LDS forms?

Expected Discussion Areas :

- Use lightning-record-edit-form which handles dependent picklists automatically
- Use getObjectInfo + getPicklistValuesByRecordType
- Use getPicklistValues to load controlling values
- Ensure correct controlling field value is set first
- Avoid hardcoding values

40 A record form loads slowly with many fields. Optimization approach?

Expected Discussion Areas :

- Load only required fields (fields attribute)
- Use layoutType="Compact" or custom field list
- Avoid unnecessary fields, tabs, rich text, etc.
- Lazy load sections / use conditional rendering
- Break large forms into sections/subcomponents
- Use LDS caching & avoid duplicate forms

Pro Tip: LDS = Faster UI + Built-in Security + Less Code + Real-time Sync!
Use it smartly to build scalable & high-performance Salesforce apps.

Parent-Child Communication

41 Difference between @api, Custom Events, and LMS?

Expected Discussion Areas :

Mechanism	Use Case	Scope	Direction	Coupling
@api	Parent → Child data/method	Direct	One-way	Tight
Custom Events	Child → Parent or Sibling	DOM Hierarchy	One-way (bubbling)	Medium
LMS (Lightning Message Service)	Unrelated Components	Application Wide	Bi-directional (pub/sub)	Loose

42 How do you send complex JSON from child to parent?

Expected Discussion Areas :

- Use CustomEvent with detail property
- Stringify JSON if needed, then parse in parent
- Pass object directly via detail
- Ensure object is serializable (no functions, no circular references)
- Example: new CustomEvent('data', { detail: { payload: obj }})
- Parent accesses via event.detail.payload

43 A custom event isn't reaching parent component. Debugging approach?

Expected Discussion Areas :

- Check if event is dispatched after component is rendered
- Ensure bubbles: true and composed: true
- Verify parent is in the DOM hierarchy
- Check for correct event name
- Use browser console logs in both child and parent
- Verify shadow DOM boundaries (need composed: true)
- Use Lightning Inspector to inspect component tree

44 How do you avoid tight coupling between components?

Expected Discussion Areas :

- Prefer LMS or Custom Events over @api for cross-component communication
- Define clear contracts (event names, payload structure)
- Use interface-like pattern (document event schema)
- Avoid direct method calls on child via template refs
- Use pub/sub model for scalability
- Follow single responsibility principle

45 How would you design communication across components placed on different tabs?

Expected Discussion Areas :

- Use Lightning Message Service (cross-tab & cross-page)
- Define a Message Channel
- Publish message from one tab/component
- Subscribe in another tab/component
- Ensures loose coupling and real-time sync
- Good for record updates, notifications, filters, etc.

46 What problems occur when passing objects via @api?

Expected Discussion Areas :

- Objects passed by reference (not deep copied)
- Child can mutate parent data accidentally
- Changes may not trigger reactivity in parent
- Large objects cause performance issues
- Circular references can break serialization
- Best practice: pass primitives or clone objects

47 How do you handle deep cloning in component communication?

Expected Discussion Areas :

- Use JSON.parse(JSON.stringify(obj)) for deep clone (simple objects)
- Use structuredClone(obj) (modern browsers)
- Use utility libraries like lodash.cloneDeep()
- Avoid cloning for large data - clone only required parts
- Ensure no functions or class instances in object

48 A child component mutates parent data accidentally. Prevention strategy?

Expected Discussion Areas :

- Pass clones instead of original objects
- Use @api setter to control incoming data
- Treat @api data as read-only in child
- Use Object.freeze() in dev mode to catch mutations
- Emit changes back to parent via events, do not mutate
- Follow immutable data patterns

49 How do you communicate between Visualforce and LWC?

Expected Discussion Areas :

- Use lightning:container in Visualforce page
- Use postMessage() API for communication
- LWC listens to window message event
- LWC sends message to VF using postMessage()
- Ensure origin validation for security
- Alternative: Use Apex as intermediary (VF ↔ Apex ↔ LWC)

50 Difference between bubbling and composed events in LWC?

Aspect	Bubbling	Composed
Definition	Event travels up the DOM tree (Parent chain)	Event crosses shadow DOM boundaries
Controls	bubbles: true	composed: true
Default	false	false
Use Case	Parent components in same DOM hierarchy	Communication outside shadow DOM boundary
Example	CustomEvent('x', {bubbles: true})	CustomEvent('x', {composed: true})

Pro Tip: Choose the right communication pattern based on scope, performance & maintainability. Loose coupling = Scalable & Easy to maintain Salesforce apps!

Performance Optimization Scenarios

51 How do you improve performance of datatable with 20k records?

Expected Discussion Areas :

- Use Server-Side Pagination / Infinite Scrolling
- Load only visible data (viewport based loading)
- Avoid loading all 20k records at once
- Use SOQL selective queries & indexed fields
- Use @AuraEnabled(cacheable=true) Apex
- Enable column virtualization (if using custom DT)
- Use lightweight custom rows / cells
- Consider search + filters to reduce dataset

52 What causes memory leaks in LWC?

Expected Discussion Areas :

- Unremoved event listeners (window, document, DOM)
- Subscriptions (LMS, Streaming API) not unsubscribed
- Timers (setInterval / setTimeout) not cleared
- Detached DOM references retained in variables
- Large objects stored in component properties
- Not cleaning up in disconnectedCallback()
- Closures capturing huge data
- Third-party libraries not destroyed properly

53 How do you lazy load components?

Expected Discussion Areas :

- Use dynamic import() for LWC components
- Load component only when needed (on demand)
- Use lightning/lazyLoader (if available) or custom wrapper
- Example: import("c/myHeavyComponent")
- Show placeholder / skeleton while loading
- Lazy load in tabs, accordions, or popups
- Reduces initial bundle size & improves TTI

54 A dashboard page containing multiple LWCs loads very slowly. Optimization plan?

Expected Discussion Areas :

- Audit each LWC for size, JS execution & re-renders
- Lazy load non-critical components
- Use @wire(cacheable=true) and LDS where possible
- Minimize Apex calls - batch / combine requests
- Use Promise.all() to parallelize independent calls
- Enable caching (Apex, Platform Cache, CDN)
- Defer below-the-fold components (Intersection Observer)
- Optimize images, static resources, and third-party libs
- Use Lightning App Builder to lazy load components

55 How do you reduce unnecessary server calls?

Expected Discussion Areas :

- Use @wire with reactive params & caching
- Use LDS instead of Apex for CRUD
- Implement client-side caching (Map, Session Storage)
- Debounce user inputs (search, filters)
- Avoid calling Apex in loops or repeatedly
- Batch multiple operations into single Apex call
- Use Platform Cache for reference data
- Use refreshApex() only when data actually changes

56 How do browser rendering cycles affect LWC performance?

Expected Discussion Areas :

- Browser: JS → Style → Layout → Paint → Composite
- Frequent DOM updates trigger reflow & repaint
- Avoid forced synchronous layouts (e.g., offsetHeight)
- Batch DOM changes in a single tick
- Use requestAnimationFrame for smooth updates
- Minimize DOM depth and complex CSS
- Use CSS transforms & opacity instead of layout changes
- Reduce heavy re-rendering & virtualize large lists

57 What are common causes of UI freezing in LWC?

Expected Discussion Areas :

- Long running JS on main thread
- Huge synchronous loops / heavy computation
- Large DOM updates / re-renders
- Too many event handlers / subscriptions
- Large data sets processed in client
- Blocking Apex calls in sequence
- Third-party scripts / libraries blocking UI
- Memory leaks leading to garbage collection pauses

58 How would you optimize image-heavy Lightning pages?

Expected Discussion Areas :

- Use responsive images (srcset, sizes)
- Compress images (WebP / optimized formats)
- Use lazy loading (loading="lazy")
- Use CDN / Static Resources with caching headers
- Avoid loading offscreen images
- Use thumbnails + on-demand full image
- Set proper width/height to avoid layout shift
- Use content delivery + image prefetch where needed

59 How do you cache Apex responses?

Expected Discussion Areas :

- Use @AuraEnabled(cacheable=true) for read-only Apex
- Leverage Lightning Data Service (LDS) cache
- Use Platform Cache (Org / Session Cache)
- Use custom client-side cache (Map, sessionStorage)
- Cache stampede prevention (TTL, cache invalidation)
- Use refreshApex() to refresh cached data
- Design cache keys carefully (user, filters, params)

60 Difference between client-side and server-side caching?

Aspect	Client-Side Caching	Server-Side Caching
Where	Browser (LWC, JS, Storage)	Server (Platform Cache, Redis)
Speed	Very Fast	Fast
Scope	Per user / per session	Shared across users
Use Case	UI state, user prefs, frequent small data	Reference data, common data, expensive queries
Data Size	Limited (Browser storage limits)	Large
Security	Per user context	Org-wide / controlled
Examples	sessionStorage, LDS cache, Map, localStorage	Platform Cache, Custom Cache (Redis, Memcached)
Invalidation	Manual / time-based	Centralized / event-based

Pro Tip: Think: Less Data, Less DOM, Less Work, Smart Caching = High Performance LWC!
Measure → Analyze → Optimize → Repeat

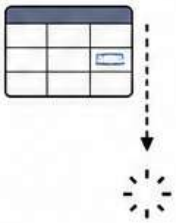
Datatable Scenarios

Scenario-Based Interview Questions (3-5 YOE)

61 How do you implement infinite scrolling in lightning-datatable?

Expected Discussion Areas :

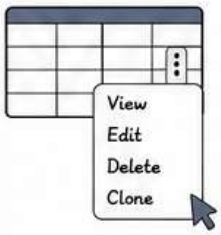
- Use onscroll or load more pattern
- Track last record id / offset
- Apex method with LIMIT and WHERE Id > lastId
- Append new data to existing list
- Show loading spinner at bottom
- Handle end of data gracefully



62 How do you implement row-level actions dynamically?

Expected Discussion Areas :

- Use type: 'action' column
- Define rowActions dynamically in JS
- Actions based on user role/permissions
- Pass row data to handleRowAction(event)
- Conditional visibility per row
- Support navigation, modal, quick actions



63 How do you handle inline editing with bulk save optimization?

Expected Discussion Areas :

- Enable editable: true on columns
- Capture draftValues
- Validate client-side before save
- Send all draft values in a single Apex call
- Use Database.update(records, false) for partial success
- Show success/error per row

64 How do you add custom cell types in datatable?

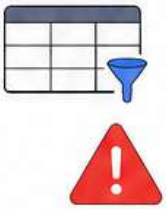
Expected Discussion Areas :

- Extend LightningDatatable
- Use static customTypes = {}
- Define template and typeAttributes
- Pass custom data via typeAttributes
- Supports images, badges, progress bars, buttons, links
- Register custom type in extended class

65 A datatable becomes unresponsive after filtering records. Why?

Expected Discussion Areas :

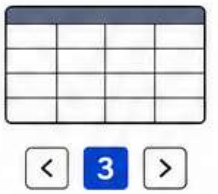
- Large dataset rendered in UI
- Too many DOM elements
- Complex cell templates
- Expensive computed properties/getters
- No pagination or lazy loading
- Memory leaks or unnecessary re-renders



66 How do you maintain pagination state during refresh?

Expected Discussion Areas :

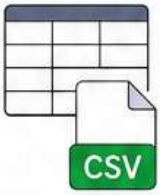
- Store current page number & page size
- Preserve sorting and filters
- Re-query data with same parameters
- Use Lightning Data Service refreshApex()
- Avoid resetting data array unnecessarily
- Maintain state in component or URL



67 How do you export datatable records into CSV/Excel?

Expected Discussion Areas :

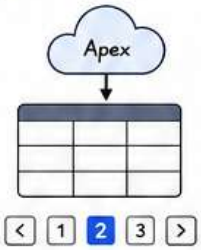
- Use selected or filtered data
- Convert JSON to CSV in JS
- Add BOM for Excel compatibility (\uFEFF)
- Create Blob and generate download link
- For large data, use Apex to generate CSV and return as ContentVersion
- Support custom headers & formatting



68 How do you implement server-side pagination?

Expected Discussion Areas :

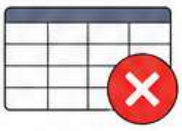
- Use LIMIT and OFFSET or seek method
- Pass pageNumber and pageSize to Apex
- Return only required records
- Total count for pagination controls
- Handle sorting and filtering in SOQL
- Improves performance for large datasets



69 What are datatable limitations in enterprise apps?

Expected Discussion Areas :

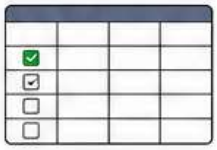
- Not ideal for 50k+ records in client-side
- No native tree/grid grouping (use custom)
- Limited custom cell UI complexity
- No built-in export for all data
- Inline edit only for supported data types
- Performance impact with too many columns



70 How do you handle selection persistence across pagination?

Expected Discussion Areas :

- Store selected record Ids in Set()
- Use getSelectedRows() + merge with existing
- Maintain selection in JS across pages
- Re-apply selected rows on data refresh
- Handle deselection properly
- Persist selection in local storage if needed



Pro Tip: For enterprise datatables, prefer server-side processing, lazy loading, and optimized data payloads for better performance and user experience.

Async & Promise Scenarios

71 Difference between Promise.all() and async/await in LWC?

Expected Discussion Areas :

Aspect	Promise.all()	async/await
Nature	Parallel execution of multiple promises	Sequential style using async functions
Readability	Harder with many chains	Easier to read (sync-like code)
Error Handling	Fails fast (rejects on first failure)	Try/Catch for granular handling
Use Case	Independent calls in parallel	Dependent calls in sequence
Return	Array of results	Single/multiple values

72 How do you handle race conditions in multiple API calls?

Expected Discussion Areas :

- Track requestId / sequence number
- Store latest requestId and ignore older responses
- Use AbortController (where supported)
- Maintain state to validate latest response
- Avoid updating UI from stale responses
- Example: Search-as-you-type scenarios

73 A spinner never stops loading in Production. Root cause?

Expected Discussion Areas :

- Missing resolve() / reject() in promise
- Unhandled exception inside try block
- Promise chain break (no return)
- setLoading(false) not executed in catch/finally
- Apex method never returns / timeout
- Multiple spinners handling same flag
- Navigation away before promise completes

74 How do you manage long-running transactions?

Expected Discussion Areas :

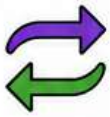
- Avoid long synchronous transactions (CPU limits)
- Break into smaller async chunks
- Use Queueable / Batch Apex for heavy jobs
- Use Platform Events / Streaming for progress
- Notify user & poll for status (status field /CDC)
- Use Continuation for callouts
- Keep UI responsive and show progress

75 How do you cancel outdated async requests?

Expected Discussion Areas :

- Use AbortController for fetch / imperative calls
- Maintain a cancel token / flag
- Increment requestId and ignore old responses
- Validate response with current requestId
- Clear timeouts / intervals in disconnectedCallback()
- Example: Live search, type-ahead, filters

76 What happens if async Apex calls return out of order?



Expected Discussion Areas :

- Older response may overwrite newer data
- UI shows stale / incorrect data
- Race condition leads to inconsistent state
- Use requestId / timestamp to accept latest only
- Disable UI / show "processing" until latest completes
- Debounce user actions to reduce requests

77 How do you handle partial failures in Promise chains?

Expected Discussion Areas :

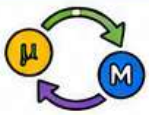
- Use Promise.allSettled() for independent calls
- Collect successes and failures separately
- Show partial data with warnings
- Log errors but don't break full UI
- Retry failed calls selectively
- Example: Load related data (Accounts, Contacts, Cases)

78 How do you debug async timing issues?

Expected Discussion Areas :

- Use console.time() / console.timeEnd()
- Add logs before/after each await
- Use unique requestId in logs
- Check Network tab for call sequence
- Use breakpoints in .then() / async functions
- Simulate slow network in DevTools
- Handle res.render asynchronoosly in tests

79 How do microtasks/macrotasks impact LWC rendering?



Expected Discussion Areas :

- Microtasks (Promises) run before next render
- DOM updates are batched in microtasks
- Too many microtasks delay rendering
- Macrotasks (setTimeout) run after render cycle
- Use microtasks for immediate UI updates
- Avoid long CPU work in microtasks
- Understand JS event loop for smooth UI

80 How would you design resilient async architecture?

Expected Discussion Areas :

- Design idempotent Apex methods
- Use retries with exponential backoff
- Implement timeout & offline handling
- Show meaningful error messages
- Use circuit breaker pattern (stop repeated failures)
- Cache critical data (LDS / Platform Cache)
- Monitor with logs, Platform Events, Analytics
- Keep UI responsive & degrade gracefully

Pro Tip: Always Plan → Handle → Recover → Inform

Great async code = Fast UI + Happy Users + Fewer Bugs!

Security & Access Control

Scenario-Based Interview Questions (3-5 YOE)

81 How do you enforce CRUD/FLS in LWC solutions?

Expected Discussion Areas :

- Use Schema describe methods
- Enforce CRUD: isCreateable(), isUpdateable(), isDeleteable(), isAccessible()
- Enforce FLS: isAccessible() on fields
- Use Lightning Data Service (LDS) where possible
- Validate again in Apex (never trust client)

82 Can users bypass frontend validations? How will you secure?

Expected Discussion Areas :

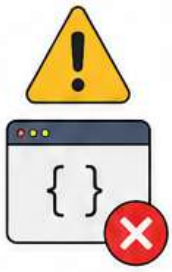
- Yes, users can bypass client-side validation
- Always validate in Apex (backend)
- Validate business rules, limits, permissions
- Use addError() for field/record level errors
- Do not rely on JavaScript validations

83 How does Locker Service impact JavaScript libraries?

Expected Discussion Areas :

- Isolation of components' JS
- Restricts access to global objects (window, document)
- No direct DOM manipulation outside component
- May block certain APIs & third-party libraries
- Improves security by namespace isolation
- Can cause library compatibility issues

84 A third-party JS library fails inside LWC. Why?



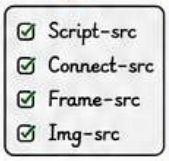
Expected Discussion Areas :

- Not compatible with Locker/Lightning Web Security (LWS)
- Direct DOM access or global variable usage
- Uses eval(), document.write(), innerHTML, etc.
- Relies on jQuery-style global selectors
- Solution: Use libraries compatible with LWS or load in iframe

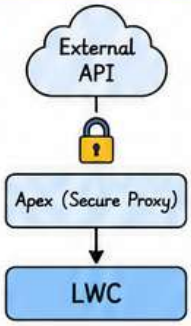
85 How do CSP restrictions affect integrations?

Expected Discussion Areas :

- CSP blocks resources from untrusted domains
- Affects external scripts, styles, iframes, images
- Need to whitelist domains in CSP Trusted Sites
- Inline scripts/styles may be blocked
- Use Named Credentials/Remote Site Settings for callouts



86 How do you securely expose external APIs to LWC?



Expected Discussion Areas :

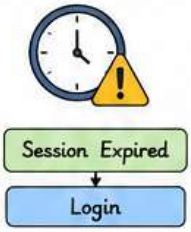
- Never call external APIs directly from LWC
- Use Apex as a secure proxy layer
- Use Named Credentials for authentication
- Enforce user permissions in Apex
- Sanitize and validate all input/output
- Avoid exposing secrets/tokens in JS

87 How do you prevent sensitive data exposure in browser console?

Expected Discussion Areas :

- Avoid console.log() in production
- Remove debug logs before deployment
- Use custom logger with environment checks
- Mask PII/Secrets in logs
- Use Platform Events for logging (not client logs)
- Enforce code review & static analysis

88 How do you handle session expiration gracefully?



Expected Discussion Areas :

- Detect expired session via error codes (e.g., 401, INVALID_SESSION_ID)
- Show user-friendly message & redirect to login
- Save unsaved data to localStorage/sessionStorage
- Use heartbeat/ping for long-running sessions
- Implement re-login flow with state restoration

89 How do you secure file uploads in LWC?

Expected Discussion Areas :

- Validate file type (whitelist)
- Validate file size (client + server side)
- Scan for malicious content (Apex/Antivirus API)
- Store in ContentVersion with proper sharing
- Enforce user access & record ownership
- Avoid exposing public URLs unnecessarily

90 Difference between with sharing and without sharing impact in LWC architecture?

with sharing	without sharing
Respects user's sharing rules	Ignores user's sharing rules
Enforces OWD, Roles, Sharing Rules, Territory, etc.	Runs in system context
Use for most business logic	Use for admin operations, scheduled jobs, integrations
More secure for multi-tenant apps	Higher risk of data overexposure
LWC calls Apex → Follows Apex class sharing model	Can expose data user shouldn't access

Pro Tip: Security is a shared responsibility — validate on client for UX, but always enforce on server for trust.